DataFrame

Constructor

`DataFrame`([data, index, columns, dtype, copy])	Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Attributes and underlying data

Axes

`DataFrame.index`	The index (row labels) of the DataFrame.
`DataFrame.columns`	The column labels of the DataFrame.

`DataFrame.dtypes`	Return the dtypes in the DataFrame.
`DataFrame.info`([verbose, buf, max_cols, ...])	Print a concise summary of a DataFrame.
`DataFrame.select_dtypes`([include, exclude])	Return a subset of the DataFrame's columns based on the column dtypes.
`DataFrame.values`	Return a Numpy representation of the DataFrame.
`DataFrame.axes`	Return a list representing the axes of the DataFrame.
`DataFrame.ndim`	Return an int representing the number of axes / array dimensions.
`DataFrame.size`	Return an int representing the number of elements in this object.
`DataFrame.shape`	Return a tuple representing the dimensionality of the DataFrame.

`DataFrame.memory_usage`([index, deep])	Return the memory usage of each column in bytes.
`DataFrame.empty`	Indicator whether Series/DataFrame is empty.
`DataFrame.set_flags`(*[, copy, ...])	Return a new object with updated flags.

Conversion

`DataFrame.astype`(dtype[, copy, errors])	Cast a pandas object to a specified dtype `dtype`.
`DataFrame.convert_dtypes`([infer_objects, ...])	Convert columns from numpy dtypes to the best dtypes that support `pd.NA`.
`DataFrame.infer_objects`([copy])	Attempt to infer better dtypes for object columns.
`DataFrame.copy`([deep])	Make a copy of this object's indices and data.
`DataFrame.to_numpy`([dtype, copy, na_value])	Convert the DataFrame to a NumPy array.

Indexing, iteration

`DataFrame.head`([n])	Return the first *n* rows.
`DataFrame.at`	Access a single value for a row/column label pair.
`DataFrame.iat`	Access a single value for a row/column pair by integer position.
`DataFrame.loc`	Access a group of rows and columns by label(s) or a boolean array.
`DataFrame.iloc`	Purely integer-location based indexing for selection by position.

`DataFrame.insert`(loc, column, value[, ...])	Insert column into DataFrame at specified location.
`DataFrame.__iter__`()	Iterate over info axis.
`DataFrame.items`()	Iterate over (column name, Series) pairs.
`DataFrame.keys`()	Get the 'info axis' (see Indexing for more).
`DataFrame.iterrows`()	Iterate over DataFrame rows as (index, Series) pairs.
`DataFrame.itertuples`([index, name])	Iterate over DataFrame rows as namedtuples.
`DataFrame.pop`(item)	Return item and drop it from DataFrame.
`DataFrame.tail`([n])	Return the last *n* rows.
`DataFrame.xs`(key[, axis, level, drop_level])	Return cross-section from the Series/DataFrame.
`DataFrame.get`(key[, default])	Get item from object for given key (ex: DataFrame column).
`DataFrame.isin`(values)	Whether each element in the DataFrame is contained in values.
`DataFrame.where`(cond[, other, inplace, ...])	Replace values where the condition is False.
`DataFrame.mask`(cond[, other, inplace, axis, ...])	Replace values where the condition is True.
`DataFrame.query`(expr, *[, parser, engine, ...])	Query the columns of a DataFrame with a boolean expression.
`DataFrame.isetitem`(loc, value)	Set the given value in the column with position *loc*.

For more information on `.at`, `.iat`, `.loc`, and `.iloc`, see the indexing documentation.

Binary operator functions

`DataFrame.__add__`(other)	Get Addition of DataFrame and other, column-wise.
`DataFrame.add`(other[, axis, level, fill_value])	Get Addition of dataframe and other, element-wise (binary operator *add*).
`DataFrame.sub`(other[, axis, level, fill_value])	Get Subtraction of dataframe and other, element-wise (binary operator *sub*).
`DataFrame.mul`(other[, axis, level, fill_value])	Get Multiplication of dataframe and other, element-wise (binary operator *mul*).
`DataFrame.div`(other[, axis, level, fill_value])	Get Floating division of dataframe and other, element-wise (binary operator *truediv*).
`DataFrame.truediv`(other[, axis, level, ...])	Get Floating division of dataframe and other, element-wise (binary operator *truediv*).
`DataFrame.floordiv`(other[, axis, level, ...])	Get Integer division of dataframe and other, element-wise (binary operator *floordiv*).
`DataFrame.mod`(other[, axis, level, fill_value])	Get Modulo of dataframe and other, element-wise (binary operator *mod*).
`DataFrame.pow`(other[, axis, level, fill_value])	Get Exponential power of dataframe and other, element-wise (binary operator *pow*).
`DataFrame.dot`(other)	Compute the matrix multiplication between the DataFrame and other.
`DataFrame.radd`(other[, axis, level, fill_value])	Get Addition of dataframe and other, element-wise (binary operator *radd*).
`DataFrame.rsub`(other[, axis, level, fill_value])	Get Subtraction of dataframe and other, element-wise (binary operator *rsub*).
`DataFrame.rmul`(other[, axis, level, fill_value])	Get Multiplication of dataframe and other, element-wise (binary operator *rmul*).
`DataFrame.rdiv`(other[, axis, level, fill_value])	Get Floating division of dataframe and other, element-wise (binary operator *rtruediv*).
`DataFrame.rtruediv`(other[, axis, level, ...])	Get Floating division of dataframe and other, element-wise (binary operator *rtruediv*).

`DataFrame.rfloordiv`(other[, axis, level, ...])	Get Integer division of dataframe and other, element-wise (binary operator *rfloordiv*).
`DataFrame.rmod`(other[, axis, level, fill_value])	Get Modulo of dataframe and other, element-wise (binary operator *rmod*).
`DataFrame.rpow`(other[, axis, level, fill_value])	Get Exponential power of dataframe and other, element-wise (binary operator *rpow*).
`DataFrame.lt`(other[, axis, level])	Get Greater than of dataframe and other, element-wise (binary operator *lt*).
`DataFrame.gt`(other[, axis, level])	Get Greater than of dataframe and other, element-wise (binary operator *gt*).
`DataFrame.le`(other[, axis, level])	Get Greater than or equal to of dataframe and other, element-wise (binary operator *le*).
`DataFrame.ge`(other[, axis, level])	Get Greater than or equal to of dataframe and other, element-wise (binary operator *ge*).
`DataFrame.ne`(other[, axis, level])	Get Not equal to of dataframe and other, element-wise (binary operator *ne*).
`DataFrame.eq`(other[, axis, level])	Get Not equal to of dataframe and other, element-wise (binary operator *eq*).
`DataFrame.combine`(other, func[, fill_value, ...])	Perform column-wise combine with another DataFrame.
`DataFrame.combine_first`(other)	Update null elements with value in the same location in *other*.

Function application, GroupBy & window

`DataFrame.apply`(func[, axis, raw, ...])	Apply a function along an axis of the DataFrame.
`DataFrame.map`(func[, na_action])	Apply a function to a Dataframe elementwise.

`DataFrame.pipe`(func, *args, **kwargs)	Apply chainable functions that expect Series or DataFrames.
`DataFrame.agg`([func, axis])	Aggregate using one or more operations over the specified axis.
`DataFrame.aggregate`([func, axis])	Aggregate using one or more operations over the specified axis.
`DataFrame.transform`(func[, axis])	Call `func` on self producing a DataFrame with the same axis shape as self.
`DataFrame.groupby`([by, level, as_index, ...])	Group DataFrame using a mapper or by a Series of columns.
`DataFrame.rolling`(window[, min_periods, ...])	Provide rolling window calculations.
`DataFrame.expanding`([min_periods, method])	Provide expanding window calculations.
`DataFrame.ewm`([com, span, halflife, alpha, ...])	Provide exponentially weighted (EW) calculations.

Computations / descriptive stats

`DataFrame.abs`()	Return a Series/DataFrame with absolute numeric value of each element.
`DataFrame.all`(*[, axis, bool_only, skipna])	Return whether all elements are True, potentially over an axis.
`DataFrame.any`(*[, axis, bool_only, skipna])	Return whether any element is True, potentially over an axis.
`DataFrame.clip`([lower, upper, axis, inplace])	Trim values at input threshold(s).
`DataFrame.corr`([method, min_periods, ...])	Compute pairwise correlation of columns, excluding NA/null values.
`DataFrame.corrwith`(other[, axis, drop, ...])	Compute pairwise correlation.

`DataFrame.count`([axis, numeric_only])	Count non-NA cells for each column or row.
`DataFrame.cov`([min_periods, ddof, numeric_only])	Compute pairwise covariance of columns, excluding NA/null values.
`DataFrame.cummax`([axis, skipna, numeric_only])	Return cumulative maximum over a DataFrame or Series axis.
`DataFrame.cummin`([axis, skipna, numeric_only])	Return cumulative minimum over a DataFrame or Series axis.
`DataFrame.cumprod`([axis, skipna, numeric_only])	Return cumulative product over a DataFrame or Series axis.
`DataFrame.cumsum`([axis, skipna, numeric_only])	Return cumulative sum over a DataFrame or Series axis.
`DataFrame.describe`([percentiles, include, ...])	Generate descriptive statistics.
`DataFrame.diff`([periods, axis])	First discrete difference of element.
`DataFrame.eval`(expr, *[, inplace])	Evaluate a string describing operations on DataFrame columns.
`DataFrame.kurt`(*[, axis, skipna, numeric_only])	Return unbiased kurtosis over requested axis.
`DataFrame.kurtosis`(*[, axis, skipna, ...])	Return unbiased kurtosis over requested axis.
`DataFrame.max`(*[, axis, skipna, numeric_only])	Return the maximum of the values over the requested axis.
`DataFrame.mean`(*[, axis, skipna, numeric_only])	Return the mean of the values over the requested axis.
`DataFrame.median`(*[, axis, skipna, numeric_only])	Return the median of the values over the requested axis.
`DataFrame.min`(*[, axis, skipna, numeric_only])	Return the minimum of the values over the requested axis.

`DataFrame.mode`([axis, numeric_only, dropna])	Get the mode(s) of each element along the selected axis.
`DataFrame.pct_change`([periods, fill_method, ...])	Fractional change between the current and a prior element.
`DataFrame.prod`(*[, axis, skipna, ...])	Return the product of the values over the requested axis.
`DataFrame.product`(*[, axis, skipna, ...])	Return the product of the values over the requested axis.
`DataFrame.quantile`([q, axis, numeric_only, ...])	Return values at the given quantile over requested axis.
`DataFrame.rank`([axis, method, numeric_only, ...])	Compute numerical data ranks (1 through n) along axis.
`DataFrame.round`([decimals])	Round numeric columns in a DataFrame to a variable number of decimal places.
`DataFrame.sem`(*[, axis, skipna, ddof, ...])	Return unbiased standard error of the mean over requested axis.
`DataFrame.skew`(*[, axis, skipna, numeric_only])	Return unbiased skew over requested axis.
`DataFrame.sum`(*[, axis, skipna, ...])	Return the sum of the values over the requested axis.
`DataFrame.std`(*[, axis, skipna, ddof, ...])	Return sample standard deviation over requested axis.
`DataFrame.var`(*[, axis, skipna, ddof, ...])	Return unbiased variance over requested axis.
`DataFrame.nunique`([axis, dropna])	Count number of distinct elements in specified axis.
`DataFrame.value_counts`([subset, normalize, ...])	Return a Series containing the frequency of each distinct row in the DataFrame.

Reindexing / selection / label manipulation

`DataFrame.add_prefix`(prefix[, axis])	Prefix labels with string *prefix*.
`DataFrame.add_suffix`(suffix[, axis])	Suffix labels with string *suffix*.
`DataFrame.align`(other[, join, axis, level, ...])	Align two objects on their axes with the specified join method.
`DataFrame.at_time`(time[, asof, axis])	Select values at particular time of day (e.g., 9:30AM).
`DataFrame.between_time`(start_time, end_time)	Select values between particular times of the day (e.g., 9:00-9:30 AM).
`DataFrame.drop`([labels, axis, index, ...])	Drop specified labels from rows or columns.
`DataFrame.drop_duplicates`([subset, keep, ...])	Return DataFrame with duplicate rows removed.
`DataFrame.duplicated`([subset, keep])	Return boolean Series denoting duplicate rows.
`DataFrame.equals`(other)	Test whether two objects contain the same elements.
`DataFrame.filter`([items, like, regex, axis])	Subset the DataFrame or Series according to the specified index labels.
`DataFrame.idxmax`([axis, skipna, numeric_only])	Return index of first occurrence of maximum over requested axis.
`DataFrame.idxmin`([axis, skipna, numeric_only])	Return index of first occurrence of minimum over requested axis.
`DataFrame.reindex`([labels, index, columns, ...])	Conform DataFrame to new index with optional filling logic.
`DataFrame.reindex_like`(other[, method, ...])	Return an object with matching indices as other object.

`DataFrame.rename`([mapper, index, columns, ...])	Rename columns or index labels.
`DataFrame.rename_axis`([mapper, index, ...])	Set the name of the axis for the index or columns.
`DataFrame.reset_index`([level, drop, ...])	Reset the index, or a level of it.
`DataFrame.sample`([n, frac, replace, ...])	Return a random sample of items from an axis of object.
`DataFrame.set_axis`(labels, *[, axis, copy])	Assign desired index to given axis.
`DataFrame.set_index`(keys, *[, drop, append, ...])	Set the DataFrame index using existing columns.
`DataFrame.take`(indices[, axis])	Return the elements in the given *positional* indices along an axis.
`DataFrame.truncate`([before, after, axis, copy])	Truncate a Series or DataFrame before and after some index value.

Missing data handling

`DataFrame.bfill`(*[, axis, inplace, limit, ...])	Fill NA/NaN values by using the next valid observation to fill the gap.
`DataFrame.dropna`(*[, axis, how, thresh, ...])	Remove missing values.
`DataFrame.ffill`(*[, axis, inplace, limit, ...])	Fill NA/NaN values by propagating the last valid observation to next valid.
`DataFrame.fillna`(value, *[, axis, inplace, ...])	Fill NA/NaN values with *value*.
`DataFrame.interpolate`([method, axis, limit, ...])	Fill NaN values using an interpolation method.
`DataFrame.isna`()	Detect missing values.
`DataFrame.isnull`()	DataFrame.isnull is an alias for DataFrame.isna.

`DataFrame.notna`()	Detect existing (non-missing) values.
`DataFrame.notnull`()	DataFrame.notnull is an alias for DataFrame.notna.
`DataFrame.replace`([to_replace, value, ...])	Replace values given in *to_replace* with *value*.

Reshaping, sorting, transposing

`DataFrame.droplevel`(level[, axis])	Return Series/DataFrame with requested index / column level(s) removed.
`DataFrame.pivot`(*, columns[, index, values])	Return reshaped DataFrame organized by given index / column values.
`DataFrame.pivot_table`([values, index, ...])	Create a spreadsheet-style pivot table as a DataFrame.
`DataFrame.reorder_levels`(order[, axis])	Rearrange index or column levels using input `order`.
`DataFrame.sort_values`(by, *[, axis, ...])	Sort by the values along either axis.
`DataFrame.sort_index`(*[, axis, level, ...])	Sort object by labels (along an axis).
`DataFrame.nlargest`(n, columns[, keep])	Return the first *n* rows ordered by *columns* in descending order.
`DataFrame.nsmallest`(n, columns[, keep])	Return the first *n* rows ordered by *columns* in ascending order.
`DataFrame.swaplevel`([i, j, axis])	Swap levels i and j in a `MultiIndex`.
`DataFrame.stack`([level, dropna, sort, ...])	Stack the prescribed level(s) from columns to index.
`DataFrame.unstack`([level, fill_value, sort])	Pivot a level of the (necessarily hierarchical) index labels.

`DataFrame.melt`([id_vars, value_vars, ...])	Unpivot DataFrame from wide to long format, optionally leaving identifiers set.
`DataFrame.explode`(column[, ignore_index])	Transform each element of a list-like to a row, replicating index values.
`DataFrame.squeeze`([axis])	Squeeze 1 dimensional axis objects into scalars.
`DataFrame.to_xarray`()	Return an xarray object from the pandas object.
`DataFrame.T`	The transpose of the DataFrame.
`DataFrame.transpose`(*args[, copy])	Transpose index and columns.

Combining / comparing / joining / merging

`DataFrame.assign`(**kwargs)	Assign new columns to a DataFrame.
`DataFrame.compare`(other[, align_axis, ...])	Compare to another DataFrame and show the differences.
`DataFrame.join`(other[, on, how, lsuffix, ...])	Join columns of another DataFrame.
`DataFrame.merge`(right[, how, on, left_on, ...])	Merge DataFrame or named Series objects with a database-style join.
`DataFrame.update`(other[, join, overwrite, ...])	Modify in place using non-NA values from another DataFrame.

Time Series-related

`DataFrame.asfreq`(freq[, method, how, ...])	Convert time series to specified frequency.
`DataFrame.asof`(where[, subset])	Return the last row(s) without any NaNs before *where*.

`DataFrame.shift`([periods, freq, axis, ...])	Shift index by desired number of periods with an optional time *freq*.
`DataFrame.first_valid_index`()	Return index for first non-missing value or None, if no value is found.
`DataFrame.last_valid_index`()	Return index for last non-missing value or None, if no value is found.
`DataFrame.resample`(rule[, closed, label, ...])	Resample time-series data.
`DataFrame.to_period`([freq, axis, copy])	Convert DataFrame from DatetimeIndex to PeriodIndex.
`DataFrame.to_timestamp`([freq, how, axis, copy])	Cast PeriodIndex to DatetimeIndex of timestamps, at *beginning* of period.
`DataFrame.tz_convert`(tz[, axis, level, copy])	Convert tz-aware axis to target time zone.
`DataFrame.tz_localize`(tz[, axis, level, ...])	Localize time zone naive index of a Series or DataFrame to target time zone.

Flags

Flags refer to attributes of the pandas object. Properties of the dataset (like the date is was recorded, the URL it was accessed from, etc.) should be stored in `DataFrame.attrs`.

`Flags`(obj, *, allows_duplicate_labels)	Flags that apply to pandas objects.

Metadata

`DataFrame.attrs` is a dictionary for storing global metadata for this DataFrame.

Warning

`DataFrame.attrs` is considered experimental and may change without warning.

`DataFrame.attrs`	Dictionary of global attributes of this dataset.

Plotting

`DataFrame.plot` is both a callable method and a namespace attribute for specific plotting methods of the form `DataFrame.plot.<kind>`.

`DataFrame.plot`([x, y, kind, ax, ...])	DataFrame plotting accessor and method

`DataFrame.plot.area`([x, y, stacked])	Draw a stacked area plot.
`DataFrame.plot.bar`([x, y, color])	Vertical bar plot.
`DataFrame.plot.barh`([x, y, color])	Make a horizontal bar plot.
`DataFrame.plot.box`([by])	Make a box plot of the DataFrame columns.
`DataFrame.plot.density`([bw_method, ind, weights])	Generate Kernel Density Estimate plot using Gaussian kernels.
`DataFrame.plot.hexbin`(x, y[, C, ...])	Generate a hexagonal binning plot.
`DataFrame.plot.hist`([by, bins])	Draw one histogram of the DataFrame's columns.
`DataFrame.plot.kde`([bw_method, ind, weights])	Generate Kernel Density Estimate plot using Gaussian kernels.
`DataFrame.plot.line`([x, y, color])	Plot Series or DataFrame as lines.
`DataFrame.plot.pie`([y])	Generate a pie plot.
`DataFrame.plot.scatter`(x, y[, s, c])	Create a scatter plot with varying marker point size and color.

`DataFrame.boxplot`([column, by, ax, ...])	Make a box plot from DataFrame columns.
`DataFrame.hist`([column, by, grid, ...])	Make a histogram of the DataFrame's columns.

Sparse accessor

Sparse-dtype specific methods and attributes are provided under the `DataFrame.sparse` accessor.

`DataFrame.sparse.density`	Ratio of non-sparse points to total (dense) data points.

`DataFrame.sparse.from_spmatrix`(data[, ...])	Create a new DataFrame from a scipy sparse matrix.
`DataFrame.sparse.to_coo`()	Return the contents of the frame as a sparse SciPy COO matrix.
`DataFrame.sparse.to_dense`()	Convert a DataFrame with sparse values to dense.

Serialization / IO / conversion

`DataFrame.from_arrow`(data)	Construct a DataFrame from a tabular Arrow object.
`DataFrame.from_dict`(data[, orient, dtype, ...])	Construct DataFrame from dict of array-like or dicts.
`DataFrame.from_records`(data[, index, ...])	Convert structured or record ndarray to DataFrame.
`DataFrame.to_orc`([path, engine, index, ...])	Write a DataFrame to the Optimized Row Columnar (ORC) format.
`DataFrame.to_parquet`([path, engine, ...])	Write a DataFrame to the binary parquet format.
`DataFrame.to_pickle`(path, *[, compression, ...])	Pickle (serialize) object to file.
`DataFrame.to_csv`([path_or_buf, sep, na_rep, ...])	Write object to a comma-separated values (csv) file.
`DataFrame.to_hdf`(path_or_buf, *, key[, ...])	Write the contained data to an HDF5 file using HDFStore.

`DataFrame.to_sql`(name, con, *[, schema, ...])	Write records stored in a DataFrame to a SQL database.
`DataFrame.to_dict`([orient, into, index])	Convert the DataFrame to a dictionary.
`DataFrame.to_excel`(excel_writer, *[, ...])	Write object to an Excel sheet.
`DataFrame.to_json`([path_or_buf, orient, ...])	Convert the object to a JSON string.
`DataFrame.to_html`([buf, columns, col_space, ...])	Render a DataFrame as an HTML table.
`DataFrame.to_feather`(path, **kwargs)	Write a DataFrame to the binary Feather format.
`DataFrame.to_latex`([buf, columns, header, ...])	Render object to a LaTeX tabular,